📖 TTSWAP 技术白皮书

1 概要

TTSWAP（Token-Token Swap）是运行在 EVM 兼容链上的去中心化做市协议（AMM）。协议通过智能合约自动撮合交易，不依赖任何中心化机构。其核心创新是**恒定价值 AMM (Constant Value AMM, CV-AMM)**：用一套 $O(1)$ 代数公式同时覆盖 Uniswap 的 CPMM、Balancer 的加权池以及 Curve 的 StableSwap 低滑点特性，且**无指数运算、无数值迭代**。

TTSWAP 的核心价值主张：

- **零无常损失的 LP 返还**：撤资按 proof 记录的历史本金比例返还代币数量，与池内当下价格无关；叠加协议级三重防御（价值锚定 + 项目方劣后缓冲 + TTS 基金）。
- **任意两币 O(1) 直达**：所有 good 挂在同一 Singleton 合约下， `buyGoodInput → buyGoodOutput` （或 `payGoodOutput → payGoodInput` ）两步完成跨币互换，流动性在协议层聚合不再被 pair 切分。
- **协议原生六方分佣**：LP / Operator / Gate / Referral / Customer / Platform 六方，合约级 `checkGoodConfig` 强制占比之和 = 100%。
- **Singleton + EIP-1153 瞬态锁 + 原生 ETH**： `buyGood` 通过 `T_GoodKey` 统一标识资产，Relayer 模式扣 `executeFee` 补偿。
- **X402 意图导向交易**： `buyGood` 与 `payGood` 在 `msg.sender != _trader` 时验证 EIP-712；Executor 代付 gas 获得固定 `executeFee` 。
- **价格翻倍解锁 + 100% 回购销毁**：TTS 代币 4C 分配 + 只有币价上涨才解锁，解锁与抛压强绑定；社区金库手续费 100% 用于二级市场回购并销毁。

2 协议特性

1. **恒定价值 AMM**：ΔV 与 Δb 全部用代数公式求解，对偶系数在 `L_Good.buyGoodInput / buyGoodOutput` （及 `payGoodInput / payGoodOutput` ）中**硬编码为常量 2**，在同一条数学框架里无缝切换 CPMM、Weighted、StableSwap。
2. **Singleton + 一币一池**： `mapping(uint256 goodid => S_GoodState) private goods` + `T_GoodKey` 寻址，保证每个资产全局只有一个池，跨币互换走价值枢纽，避免 N^2 的 pair 碎片化。 `T_GoodKey` 的 `(ercType, contractAddress, id)` 三元组为后续**股票化代币（ERC-1155）与预测市场结果份额（ERC-6909）等多 id 资产**预留了寻址扩展位（详见 §4.2.1）。
3. **流动性放大（enpower）**：通过价格分层,收缩流动性区间,放大流动性。
4. **零 IL 协议级保障**： `L_Good.disinvestGood` 按 `proofInvest × shares / proofShares` 返还原始数量 + liquid fee 分润；三重防御兜底。
5. **EIP-1153 瞬态存储**： `L_Transient` 用 `tstore/tload` 实现重入锁（ `guardedEntry` ： `lock 0→2` 独立调用、保留 `1→2` 嵌套层级， `lock > 1` 即回滚 `TTSwapError(3)` ）与 ETH `msg.value` 深度计数； `multicall` 通过 `multicallEntry` 将锁置为 1，批量子调用共享同一 ETH 预算。
6. **协议原生六方分佣**： `L_Good.allocateFee` 把每笔撤资 profit 按 `liquidFee / operatorFee / gateFee / referFee / customerFee / platformFee` 六路自动分发。
7. **原生 ETH 支持**： `L_Currency.NATIVE = address(1)` ，无需 WETH 包装，未用完的 ETH 在交易结束自动退回。
8. **Proof of Investment + LP 挖矿**：LP 投资即自动 `L_Proof.stake(TTS, investor, value)` ，一份投资 = 交易凭证 + 治理代币算力 + 收益凭证。
9. **价格翻倍解锁代币经济**：4C 分配（Founder/Partner/Value/Capital 20/12/44/24%）+ 单次解锁上限 1/20, 1/14, 1/12, 1/8 + 阈值由 `ishigher` 链上判定，**完全链上化**。
10. **X402 交易协议**： `buyGood` / `payGood` 签名与执行分离，Executor 代付 gas 并从输出代币（或同币路径的输入代币）扣 `executeFee` 。
11. **可升级 Proxy 部署**： `TTSwap_Market_Proxy` / `TTSwap_Token_Proxy` 持有状态， `implementation` 可热升级；DAO Admin 可 `disableUpgrade` 永久锁死升级权。

3 恒定价值 AMM 模型

3.1 核心交易公式（v2.0.0： `buyGoodInput` / `buyGoodOutput` ）

当用户用 Δa 数量的 Token A 兑换 Token B，协议分两步走。单笔大额 swap **按池深度的 1% 分片迭代**，每片套用同一代数核，并在结束时用 `safeLine` 校验库存上下界。

单步价值映射（输入侧， `buyGoodInput` ）

$$\Delta V_i = \frac{2 \cdot V \cdot \Delta q_i}{2 \cdot Q + \Delta q_i}$$

单步数量映射（输出侧， `buyGoodOutput` ）

$$\Delta q_i = \frac{2 \cdot Q \cdot \Delta V_i}{2 \cdot V + \Delta V_i}$$

对应 `L_Good.buyGoodInput / buyGoodOutput` 内层循环（每步最多处理 `current_quantity / 100` 或 `current_value / 100` ）：

```
// 输入侧：卖 Token A，累积转移价值 ΔV
value = (2 * _swapParam * current_value)
      / (2 * current_quantity + _swapParam);
swapTemp.get += value;
swapTemp.current_quantity += _swapParam;

// 输出侧：买 Token B，累积输出数量 Δq
quantity = (2 * _swapParam * current_quantity)
         / (2 * current_value + _swapParam);
swapTemp.get += quantity;
swapTemp.current_quantity -= quantity;
```

安全线约束：

- 输入侧： `current_quantity ≤ getSafeLineUpper(currentState.amount0 + goodConfig.amount1())` ，否则 `TTSwapError(55)` ；
- 输出侧： `current_quantity ≥ getSafeLineLower(currentState.amount0 + goodConfig.amount1())` ，否则 `TTSwapError(56)` 。

参数说明（与代码字段一一对应）：

符号	含义	代码字段
V	协议中该 Token 的总价值（投资价值）	`investState.amount1()`
Q	协议中该 Token 的当前数量（含累积手续费 + 虚拟量）	`currentState.amount1()`
Δa	扣除 `sellFee` 后入池的净输入量	`buyGoodInput` 的 `_swapParam`
ΔV	本次交易产生的"转移价值"（分片累加）	`buyGoodInput` 返回值 `amount1`
Δb	用户获得的 Token B 毛数量（扣 `buyFee` 前）	`buyGoodOutput` 返回值 `amount1`
2	公式中的固定系数	硬编码于 `L_Good`

> **设计哲学**：交换的本质是**用户愿意承担的数量失衡**与**协议价值守恒**的博弈；分片迭代 + `safeLine` 在保持 $O(1)$ 单步代数的同时，抑制大额单笔对池深度的瞬时冲击。

3.3 与主流 AMM 的数学等价性（形式化证明见附录 A–D）

- **CPMM（Uniswap V2）**：$V_A = V_B$ 时，公式严格退化为 $x \cdot y = k$。
- **Weighted Pool（Balancer）**：$V_A \neq V_B$ 时，Spot Price 与 Balancer 加权池 $SP = \frac{Q_A W_B}{Q_B W_A}$ **完全一致**（以 V 映射 W），但**无需指数运算**。

3.4 路径无关性 / 闭环性

代码逻辑经严格代数推导满足：

- **反向兑换闭环**（附录 I）：$A \to B$ 后 $B \to A$，无手续费情形下严格还原（。
- **拆分交易误差**（附录 J）：单笔 Δa 与拆为 $x_1 + x_2$ 的输出差异为 $O\left(\left(\frac{\Delta a}{Q_A}\right)^2 \cdot \frac{V_A^2 - V_B^2}{V_B^2}\right)$，工程可忽略。
- **三角闭环**（附录 K）：$A \to B \to C \to A$ 严格回到初始状态。

3.5 交易后状态更新（ `buyGood` 写回顺序）

`buyGood` 调用链： `good1change = g1.buyGoodInput(amount0)` → `good2change = g2.buyGoodOutput(good1change.amount1())` 。

侧	`currentState.amount0` （actual 入池）	`currentState.amount1` （current quantity）	`investState`
Token A（输入）	+ `sellFee`	+ `swapParam` （净输入 + 费）	不变

侧	currentState.amount0 （actual 入池）	currentState.amount1 （current quantity）	investState
Token B（输出）	+ buyFee	− outputGross （含费扣减前的池侧变动）	不变

每次状态变更路径结束后，相关 good 调用 goodConfig.updateRunBlockConfig() 刷新 **run-block 槽**（ block.number % 4095 ， bits 196–185）。 investGood 更新所投 good； buyGood 仅更新**输出侧** good2； payGood 在流程开头先更新 good2。同一 good 在同一 block 槽内二次写入会 revert TTSwapError(46) 。 investState 仅在投资 / 撤资 / 福利注入时改动。

3.6 Run-Block 防重放（ updateRunBlockConfig ）

v2.0.0 用 **区块槽**（非时间戳）限制单池单区块内的重复状态写入：

```
// L_GoodConfig.updateRunBlockConfig
uint256 slot = block.number % 4095;
if (config.getRunBlockConfig() == slot) revert TTSwapError(46);
return (config & ~mask) | (slot << 185);
```

_checkGoodActive 在入口亦校验 getRunBlockConfig() == block.number % 4095 ， 提前阻断 busy slot。

入口	更新 run-block 的 good	集成注意
investGood	所投 good	同 block 内不可对同一 good 连续 invest
buyGood	**仅 good2（输出）**	同 block 对同一输出币二次 buy 会失败；输入币不受此次写入影响
payGood	**good2（输出）**，在 AMM 计算前写入	同 block 对同一输出币二次 pay 会失败

聚合器 / 测试 harness 应在连续操作间 vm.roll 递增 block.number ， 或使用 multicall 将多步编排到同一交易但避开同一 good 的重复槽。

4 Token 模型

4.1 Token 的六维状态

属性	含义	代码字段
市场价值 V	池内权重 / 投资价值	investState.amount1()
当前数量 Q	当前数量（含手续费 + 虚拟量）	currentState.amount1()
实际入池量 I	累计实际代币入池	currentState.amount0()
份额 S	LP 总份额	investState.amount0()
虚拟量	杠杆放大后的虚拟深度	goodConfig.amount1() （低 128 位）
配置	费率 / 分佣 / 标志位	goodConfig （高 128 位）

4.2 T_GoodKey 与 Good ID

v2.0.0 所有链上入口以 T_GoodKey 标识资产，不再直接传 address ：

```
struct T_GoodKey {
    uint8 ercType;          // 1: ERC20, 2: ERC1155, 3: ERC6909（当前仅 ERC20 已启用）
    address contractAddress;
    uint256 id;             // ERC20 固定为 0
}
```

goodid = T_GoodKey.toId() ： ERC20 / 原生 ETH 取 uint160(contractAddress) ； ERC-1155 / ERC-6909 取 keccak256(packed, id) （其中 packed = (ercType << 160) | uint160(contractAddress) ）。 goods 映射以 uint256 goodid 为键。

> **当前状态**：v2.0.0 仅启用 `ercType == 1`（ERC20 / 原生 ETH）；`ercType == 2/3` 的 `toId` / `transferFrom` 分支在 `T_GoodKey.sol` 中保留但暂时 `revert TTSwapError(42)`。

4.2.1 为什么是 `(ercType, contractAddress, id)` 三元组——面向资产拓展的寻址层

`T_GoodKey` 把"资产标准 + 合约 + 子 id"三者解耦，是协议**面向后续资产类别拓展**的核心设计点。`id` 字段对 ERC20 恒为 0，但对多 id 资产标准（ERC-1155 / ERC-6909）则用来区分**同一合约下的不同标的**。这正好命中两类"一合约多 id"资产：

拓展方向	自然映射	`T_GoodKey` 编码	一币一池效果
股票化代币（Tokenized Equity / RWA）	一个发行合约托管多支股票，`id` = 单只股票标识	`ercType=2`（ERC-1155），`contractAddress=发行合约`，`id=股票编号`	每支股票在 Singleton 下独占一个货架，与现有 USDT / ETH 货架共用同一结算台
预测市场合约（Prediction Market）	一个市场合约下每个结果（YES / NO / 多结果）是一个 outcome share	`ercType=3`（ERC-6909），`contractAddress=市场合约`，`id=结果编号`	每个 outcome 份额是独立 good，可直接用 `buyGood` 与任意价值 Token 互换

设计含义：

- **零核心改动接入新资产类**：CV-AMM、`safeLine`、六方分佣、`disinvestProof` 全部以 `uint256 goodid` 为操作单元，与底层是 ERC20 还是 ERC1155/6909 **无关**。新资产类只需在 `T_GoodKey.toId()` / `transferFrom()` 打开对应 `ercType` 分支即可挂载，业务逻辑层无需改写。
- **`goodConfig.getContractType` 协同**：`contractType` 字段（`L_GoodConfig.getContractType`）可在配置层标注资产类别（普通代币 / 股票 / 预测结果等），供前端与风控按类区分展示与限额，而不污染 `goodid` 寻址。
- **跨标的直达 O(1) 不变**：股票化代币、预测份额一旦上架，即可与 USDT / ETH 等价值 Token 走 `buyGoodInput → buyGoodOutput` 两步直达，无需为每个新标的单独建 pair——这正是 Singleton 一币一池对 RWA / 预测市场的天然适配。

4.3 Token 分类

分类	标志位	说明
价值 Token (Value)	`isvaluegood() = true`（bit 255）	协议锚定资产（USDT / ETH / WBTC 等）。`initGood` 后由 **Market Admin** 通过 `modifyGoodByAdmin` 设置 `isValueGood` 位。
普通 Token (Normal)	`isvaluegood() = false`	长尾资产。交易受 `safeLine` 上下界约束（`TTSwapError(55/56)`），防止池深度被瞬时抽干。

> 所有 good 均通过统一的 `initGood` 单币启动，初始配置固定为 `L_GoodConfigLibrary.setInitialConfig()`，费率与分佣默认值见附录 E。

4.4 Token 配置：256 位 Bit-Pack 存储

`goodConfig` 是一个 `uint256`：高 128 位为静态/半静态配置，低 128 位（`amount1`）记录运行时虚拟量增量。

三层修改权限（v2.0.0）

区段	Bit 范围	谁能改	典型字段
超级管理员区	255–247	Market Admin（`modifyGoodByAdmin`）	`isValueGood`、ERC 类型预留
管理员区	246–191	Market Manager（`modifyGoodByManager`）	`isFreeze`、`isVerified`、`isPromise`、六方分佣、`limitPower`、`safeLine`
用户区	166–128	Good Owner（`modifyGoodByGoodOwner`）	`power`、交易费率、`disinvestChips`

市场可调整区（Manager 写入，须 `checkGoodConfig`）

Bit	配置	位数	访问函数	单位	默认
255	价值 Token 标志	1	`isvaluegood()`	bool	0
246	冻结标志	1	`isFreeze()`	bool	0

Bit	配置	位数	访问函数	单位	默认
245	已验证标志	1	`isVerified()`	bool	0
244	承诺申请标志	1	`isPromised()`	bool	0
243–241	LP 分佣	3	`getLiquidFee()`	`× 0.1`	6
240–237	Operator 分佣	4	`getOperatorFee()`	`× 0.02`	1
236–234	Gate 分佣	3	`getGateFee()`	`× 0.04`	5
233–229	Referral 分佣	5	`getReferFee()`	`× 0.01`	8
228–224	Customer 分佣	5	`getCustomerFee()`	`× 0.01`	8
223–219	Platform 分佣	5	`getPlatformFee128()`	`× 0.01`	2
218–214	最大放大倍数	5	`getLimitPower()`	`× 100`	1
213–204	安全线	10	`getSafeLineUpper/Lower()`	raw	80

> 六项分佣之和由 `checkGoodConfig()` 强制等于 100%。

用户可调整区（Good Owner 通过 `modifyGoodByGoodOwner` 修改）

Bit	配置	位数	访问函数	单位	默认
166–162	放大倍数 power	5	`getPower()`	`× 100`	1
161–154	撤资切片数	8	`getDisinvestChips()`	`amount/存储值×4`	10
153–148	投资费率	6	`getInvestFee()`	万分之一	8
147–142	撤资费率	6	`getDisinvestFee()`	万分之一	8
141–135	买入费率	7	`getBuyFee()`	万分之一	8
134–128	卖出费率	7	`getSellFee()`	万分之一	8

4.5 Singleton + 一币一池：超市货架的直觉类比

TTSWAP 把所有 Token 的池子挂在同一个 `TTSwap_Market` 合约的 `mapping(uint256 goodid => S_GoodState) private goods` 下，以 `T_GoodKey` 寻址。**每个资产对应一个货架，整间超市只有一个结算台**。

传统 DEX：每两种商品开一家"专卖店"

以 Uniswap V2 为例，想把"面包"换成"牛奶"，前提是必须先有人开了一家"面包 ↔ 牛奶"专卖店（pair）。如果没有，你只能："面包 → 现金（ETH/USDC）→ 牛奶"——两次走进两家不同的店，还要付两次路费。

超市世界类比：**每两个商品之间都要单独开一个兑换窗口**。

商品数量	需要开的兑换窗口数（pair）
3 种（面包、牛奶、鸡蛋）	3
10 种	45
100 种	4,950
N 种	$\frac{N(N-1)}{2}$

问题：

- 每个窗口都要**独立备货**（双边流动性），面包和牛奶在"面包 ↔ 牛奶"窗口的库存与"面包 ↔ 鸡蛋"窗口的库存**完全不相通**；
- 新上架的长尾商品通常没人开窗口，等于**根本无法交易**；
- 用户必须学会路由：去哪家店、经几家店、滑点几次、Gas 付几次。

TTSWAP：一个货架，任意兑换

TTSWAP 把所有商品挂在同一间超市的**同一条货架**上。每个商品有自己的货位（V/Q 四元组），但所有货位共用同一个**结算台**——协议内部的"价值枢纽"。

面包换牛奶的兑换过程（对应 `buyGood` 的 `buyGoodInput → buyGoodOutput` 两步）：

1. **结算台先给面包定价**：称一下你的面包有多重、当前货架上有多少面包（Q_A）、这批面包对应多少总价值（V_A），计算出面包带来的"购买力" ΔV；
2. **再用这笔购买力去兑牛奶**：看一眼牛奶货架上有多少瓶（Q_B）、对应多少总价值（V_B），按公式算出你能拿走多少牛奶 Δb。

整个过程在**一次链上调用**内完成，你从来没离开过这间超市，也没经过任何中间商品。

同一货架的两大直觉好处

1. **任意两商品直达 O(1)**
   无论是"面包换牛奶"还是"鸡蛋换黄豆"，都是同样的两步操作：商品$_A \to \Delta V \to$ 商品$_B$。不需要 `面包 → 现金 → 牛奶` 这样的多跳。
   N 种商品之间 $\frac{N(N-1)}{2}$ 对兑换路径，**在 TTSWAP 里共用同一个结算台**。
2. **流动性聚合**
   超市新进一批面包时，这批面包的流动性能被"所有和面包相关的兑换"共用，而不是只服务于"面包 ↔ 牛奶"这一个窗口。这就是 §1 提到的"**任意两币 O(1) 直达**"与"**流动性在协议层聚合不再被 pair 切分**"的真实含义。

与传统超市的对照

维度	传统 DEX（每对一店）	TTSWAP（一间超市）
商品之间的兑换路径	N 种商品需 $\frac{N(N-1)}{2}$ 个 pair	每种商品只需一个货架
新商品上架	需要额外资金建 pair，长尾商品无人建	直接挂货架即可被所有人兑换
流动性利用率	被切分到每对 pair，各自为战	所有货架共享同一结算台的流动性深度
用户路由复杂度	需找路径（单跳 / 多跳 / 聚合器）	永远是"两步走"
Gas 成本	多次跨合约调用	单次调用，Singleton 内完成

一句话总结：`mapping(uint256 goodid => S_GoodState) private goods + T_GoodKey.toId()` 这一组 Solidity 结构，把 DeFi 世界的"N 家专卖店"压缩成了 TTSWAP 的"一间大型商超"——**商品和商品之间不再需要两两配对，任何一个商品都能换任意另一个商品**。

5 流动性管理：投资与撤资

5.1 投资证明 (Proof of Investment)

LP 每次提供流动性时，协议在 `mapping(uint256 proofid => S_ProofState) private proofs` 中记录：

```
struct S_ProofState {
    uint256 currentgood;   // 投资标的 goodid
    uint256 shares;        // amount0: 份额
    uint256 state;         // amount0: 虚拟价值, amount1: 实际价值
    uint256 invest;        // amount0: 虚拟数量, amount1: 实际数量
}
```

`proofid = keccak256(abi.encode(owner, currentgood))` 。

5.2 投资路径（v2.0.0 单币入口）

函数	使用场景	权限	输入
`initGood`	单币启动新 Token，自定初始价格	任何用户	`T_GoodKey + _initial` （amount0=价值, amount1=数量）
`investGood`	向已存在池子追加单币流动性	任何用户	`T_GoodKey + _invest.amount1` （代币数量）

`initGood` 约束：

- `_initial.amount1` $\in [500{,}000, 2^{109}]$（`TTSwapError(36)`）；
- `_initial.amount0` $\in [5 \times 10^{14}, 2^{109}]$（`TTSwapError(35)`）；
- 初始 `goodConfig` **固定**为 `setInitialConfig()`，不可在 init 时自定义；
- 调用者自动成为 `owner`，并创建首笔 proof；
- **`initGood` 不调用 `L_Proof.stake`**：首笔流动性仅铸造 proof 份额，TTS 算力在后续 `investGood`（且 good 标记 `isPromised`）时才开始累计。

`investGood` 核心流程：

1. `enpower = goods[goodid].getInvestPower()`（按池内 Q/I 比率动态缩放，上限 `getPower()`）；
2. 转入 `_invest.amount1` 代币；
3. `L_Good.investGood(quantity, result, enpower)`：
   - `investFeeQuantity = getInvestFee(quantity)`；
   - `investQuantity = (quantity − fee) × enpower / 100`（虚拟放大）；
   - `investValue = poolPrice × investQuantity`，经 `getInvestThreshold` 截断；
   - `investShare` 按份额比例铸造；
4. 写回池：`currentState`、`investState`、`goodConfig.amount1`（虚拟量）；
5. 更新 proof；若 `goodConfig.isPromised()` 为真，则 `L_Proof.stake(TTS, msg.sender, investvalue)` 并按比例记入 proof `shares.amount1`；否则 stake 量为 0。

> v2.0.0 移除了 owner 自报价分支（`isInvestBlocked` / `_invest.amount0 > 0`），追加流动性一律按池内现价自动计价。

5.3 撤资路径

`disinvestProof(_proofid, _goodshares, _gate, _trader, signature)` 完成以下链式操作：

1. **权属校验**：`S_ProofKey(_trader, currentgood).toId() == _proofid`；
2. **承诺锁**：若 `isPromised() && owner == _trader`，revert `TTSwapError(40)`（首发承诺期内 owner 不可撤资）；
3. **调用 `L_Good.disinvestGood`**：
   - `actualDisinvestQuantity = proofInvest1 × _goodshares / proofShares0`（**按 proof 历史比例返还**）；
   - 计算 profit + `getDisinvestFee` + `allocateFee` 六路分润；
4. **转账**：`toGoodKey().safeTransfer(msg.sender, commission − 1)` 把累积佣金 + 本金发给 LP；
5. **同步 TTS 解押**：仅当 `disinvestTTSValue > 0` 时调用 `L_Proof.unstake(TTS, msg.sender, disinvestTTSValue)`（`initGood` 未质押时撤资可跳过 unstake）。

> `signature` 参数保留 ABI 兼容，**不验证**；`_trader` 必须等于 `msg.sender`。

5.4 抵御无常损失的三重防御

TTSWAP 的 LP 在撤资时按**历史本金数量**取回代币，与池内当下价格无关，这等于把 IL 外部化为"最后出场者风险"。协议通过三重防御把该风险收敛：

1. **协议级价值锚定**：`disinvestGood` 代码路径严格按 proof 历史计算，LP 本位数量有形式化保证。
2. **项目方劣后缓冲**：meta/value good 的初始流动性由发行方注入，事实上构成"劣后级本金"，在极端行情下先行吸收波动。
3. **TTS 治理代币补偿基金**：极端缺口由 TTS 拍卖 / 释放兜底，由 DAO Admin `mint/burn` 权限作为最后兜底（白皮书 §12.7）。

5.5 撤资切片（Disinvest Chips）

`getDisinvestChips(investState.amount1()) ≥ disinvestvalue.amount0()`，限制单次撤资价值比例（由 Token Owner 配置）。超过上限的单笔撤资会 revert `TTSwapError(26)` 或 `TTSwapError(27)`。

5.6 首发承诺机制（Promise + `refreshPromise`）

Good Owner 可在 `goodConfig` 设置 `isPromised` 标志位（bit 244），配合 `refreshPromise(proofid)` 使用：

- 调用方**必须是 proof 所有者本人**：`S_ProofKey(msg.sender, currentgood).toId() == proofid`；
- 若 `goodConfig.isPromised() && owner == msg.sender`，协议广播 `e_getPromiseProof(currentgood, proofid)` 事件，供链下 Launchpad / IDO 系统消费；
- `refreshPromise` **不支持 relayer、不接受 EIP-712 签名**；
- 承诺期内 owner 调用 `disinvestProof` 将被 `TTSwapError(40)` 阻断。

5.7 合约参数与限制（实测值）

参数	限制值	源码位置	含义
初始数量下限	500,000	`initGood._initial.amount1() < 500000`	`TTSwapError(36)`
初始数量上限	2^{109}	`initGood._initial.amount1() > 2**109`	`TTSwapError(36)`
初始自定价值下限	5×10^{14}	`initGood._initial.amount0() < 5e14`	`TTSwapError(35)`
初始自定价值上限	2^{109}	`initGood._initial.amount0() > 2**109`	`TTSwapError(35)`
追投最小价值	10^{12}	`investGood` 内 `investValue < 1e12`	`TTSwapError(38)`
最小交易价值	10^{8}	`buyGood` 检查 `good1change.amount1() < 1e8`	`TTSwapError(14)`
数量溢出保护	2^{109}	`currentState.amount1() + delta > 2**109`	`TTSwapError(18)`
Executor 补偿	**50,000,000,000** = 5×10^{10}	`executeFee` 常量	12 位精度稳定币下 = **0.05 USDT**

5.8 安全线约束

`safeLine` 定义池深度的可交易区间：

- **卖入侧**（`buyGoodInput` / `payGoodInput`）：`Q` 不得超过 `getSafeLineUpper(I + virtual)`，否则 `TTSwapError(55)`；
- **买出侧**（`buyGoodOutput` / `payGoodOutput`）：`Q` 不得低于 `getSafeLineLower(I + virtual)`，否则 `TTSwapError(56)`。

`safeLine` 在代码中实现为**两个独立 8 位字段**：`getSafeLineUpper`（bits 219–212）与 `getSafeLineLower`（bits 211–204）。**默认存储值为 0**，此时上 / 下界直接回退为基准库存 `I + virtual`（不额外约束）；非零存储值 v 时，界 $= v \times (I + \text{virtual})/100$，由 Market Manager 通过 `modifyGoodByManager` 配置。

```
// buyGoodInput / payGoodInput 结束前
if (current_quantity > goodConfig.getSafeLineUpper(
        currentState.amount0() + goodConfig.amount1()
    )) revert TTSwapError(55);

// buyGoodOutput / payGoodOutput 结束前
if (current_quantity < goodConfig.getSafeLineLower(
        currentState.amount0() + goodConfig.amount1()
    )) revert TTSwapError(56);
```

语义：**普通代币的卖出/买入自由度受池深度安全区间约束**，防止大额单笔将池子瞬时推至不可持续状态。

5.9 独立流动性定义

- 任意 Token 通过 `initGood` **单币启动**，无需配对价值 Token；
- 通过 `investGood` 单边追加流动性；
- 撤资按本 proof 份额单独结算；
- 价格由该池自身的 $(V, Q, I, power)$ 决定；
- `safeLine` 按池独立生效。

5.10 价格约束统一化

(A) 投资：v2 追加流动性一律按池内价自动计价（`investGood` 无自报价分支）。

(B) 交易滑点：

- `buyGood` exact-in：`good2change.amount1() >= _swapQuantity.amount1()` 否则 `TTSwapError(15)`。

(C) 池深度健康度：`safeLine` 上下界（见 §5.8）。

5.11 单边流动性可行性：安全线如何防止池子被抽干

> **核心命题**：v2 用 `safeLine` 定义每个池的可交易深度区间，大额卖压/买压在触及边界时 revert，迫使市场通过多笔小单或反向套利完成价格发现。

5.11.1 安全线约束

对所有 good（价值 Token 与普通 Token 均适用），swap 路径在 `buyGoodInput` / `buyGoodOutput`（及 pay 逆路径）分片循环结束后校验：

$$Q_{\text{lower}} \;\le\; Q \;\le\; Q_{\text{upper}}$$

其中上下界由 `getSafeLineLower/Upper(currentState.amount0 + goodConfig.amount1())` 计算。越界分别 revert `TTSwapError(56)` / `TTSwapError(55)`。

5.11.2 攻击路径被阻断

攻击者持续卖出普通代币 `$JUNK` 套取 USDT 时：

1. 每笔卖单推高 $JUNK$ 的 Q；
2. 当 $Q > Q_{\text{upper}}$，`buyGoodInput` revert `TTSwapError(55)`；
3. 必须等待买盘拉回 Q 或 LP 撤资释放空间后，才能继续卖入。

价值 Token vs 普通 Token：两者共用同一套 `safeLine` 机制；差异在于价值 Token 通常由 Admin 提权（`isValueGood`），池深更大、流动性更充裕。

5.11.3 超市进货的日常类比

> 供货商给社区超市供了 **1,000 瓶矿泉水**，进货价 **10 元 / 瓶**。
> 超市的货架（=池子 Q）最多放得下这 1,000 瓶；如果没有任何人来买，供货商却想再送来 500 瓶，超市老板一定会拒收——**因为货架放不下，放进去的水就只能烂在仓库，还要挤占其它商品的位置**。
> 只有当市场真的开始买水（货架出现 200 瓶空位），供货商才能继续补 200 瓶。**买家先出手，卖家的"继续卖"才有空间**。

TTSWAP 对普通代币的处理与超市完全同构：

超市世界	TTSWAP 对应物
货架容量	$I + \text{virtual quantity}$（LP 支撑的库存上限）
供货商来补货	用户卖出普通代币（`buyGood` A → B，A 为普通代币）
消费者来买水	用户买入普通代币（`buyGood` B → A）
老板拒收超量进货	`TTSwapError(55)` revert
其它货架上的牛奶、饼干	协议里的其它价值资产（USDT / ETH …）

如果超市老板来者不拒，供货商会一直往里倒货 → 超市的现金、货架、其它商品全部被 "烂水" 挤走，这是 DeFi 里典型的"普通代币反向抽干"攻击。TTSWAP 用 `safeLine` 上下界把这条路径**从合约层直接关掉**。

5.11.4 价值代币 vs 普通代币

维度	价值代币（Value Good）	普通代币（Normal Good）
定义	`isvaluegood() == true`	`isvaluegood() == false`
典型例子	USDT / ETH / WBTC	新项目代币、长尾资产
Admin 提权	`modifyGoodByAdmin` 设置 bit 255	默认不设置
流动性追加	`investGood` 按池内价线性计价	同左，共用 `L_Good.investGood`
交易深度约束	`safeLine` 上下界	同左

v2.0.0 投资路径不再区分价值/普通分支公式，统一走 `L_Good.investGood + getInvestThreshold`。

5.11.5 外部价格暴跌时的价格修复闭环

> 极端场景：某个普通代币在 CEX / 其他 DEX 的价格暴跌 50%，套利者涌向 TTSWAP 卖出。

协议按以下顺序响应，形成**逐步收敛、不崩盘**的闭环：

1. **卖压上涨 → safeLine 逼近**：分批卖单被成交，Q 快速逼近 Q_{upper}。
2. **达到上限 → 协议停止继续成交**：下一笔 `buyGood` A→B 会被 `TTSwapError(55)` 阻断。

3. **外部套利者反向介入**：价差拉大后，套利者在相反方向买入，把池内价格往真实价格拉回。
4. **LP 撤资释放空间**： `disinvestProof` 按 proof 历史数量返还，与当下池价解耦（§5.4）。
5. **Owner 追加流动性**：通过 `investGood` 按池内现价补仓，为下一轮交易腾出深度。

这 5 步本质上是把"外部暴跌"分摊成多个小步的价格发现过程，而不是让协议瞬间被清空。**LP 的损失来自时间，不来自瞬时滑点放大**。

5.11.6 为什么这样设计不叫"限制交易"而叫"安全边界"

- **价值代币之间的 swap**（例如 USDT ↔ ETH）与 **普通代币买入**均正常进行；
- **大额卖入/买出**在逼近 `safeLine` 边界时才受限；
- 用户随时可以**撤资**（`disinvestProof`）或**福利注入**（`goodWelfare`）主动调整池深度。

因此 `safeLine` 是一条**防止外部价格崩溃穿透到协议资产负债表**的安全带，而不是对正常交易的限流阀。

5.11.7 最终小结

TTSWAP 的单边流动性可行，依赖四个代码层机制：

1. **`safeLine` 上下界**（`TTSwapError(55/56)`）：限制单笔及累计 swap 对池深度的冲击；
2. **分片迭代 swap**（`buyGoodInput/Output` 、 `payGoodInput/Output`）：大额交易按 1% 深度分步定价；
3. **Proof 历史记账 + 三重防御**（§5.4）：LP 撤资返还量与当下池价解耦；
4. **动态 `getInvestPower`**：按 Q/I 比率缩放追加流动性的杠杆倍数。

5.12 安全攻击面（代码层防御映射）

攻击	防御机制	代码位置
低流动性价格操纵	最小初始值 / 初始价值下限	`initGood`
三明治（Sandwich）	EIP-712 `external_info` deadline + `minOut`	`buyGood`
库存抽干 / 超买	`safeLine` 上下界 `TTSwapError(55/56)`	`buyGoodInput` / `buyGoodOutput` / `payGoodInput` / `payGoodOutput`
重入攻击	EIP-1153 瞬态锁 `guardedEntry` + `multicallEntry`	`L_Transient`
同 block 双写	run-block 槽 `TTSwapError(46)`	`updateRunBlockConfig` / `_checkGoodActive`
佣金双拿	`referral = gate ? 0 : referral + isBan()` 归零	`disinvestProof`
福利注入超额	`currentState.amount0() + welfare > 2**109` 回滚	`goodWelfare`
建池抢占	`if (goods[goodid].owner != 0) revert TTSwapError(5)`	`initGood`
首发承诺期撤资	`isPromised && owner` 阻断	`disinvestProof`

6 交易函数

TTSWAP v2.0.0 提供 **`buyGood` （Exact-In）**、 **`payGood` （Exact-Out / 同币转账）** 与 **`multicall` （批量编排）** 三类交易入口。

6.1 `buyGood` （Exact-In 买入）

```
function buyGood(
    T_GoodKey memory _goodKey1,   // 卖出代币（输入 good）
    T_GoodKey memory _goodKey2,   // 买入代币（输出 good）
    uint256 _swapQuantity,        // amount0: 输入量, amount1: 最小毛输出（滑点下限）
    address _recipient,           // 推荐人：非零且 ≠ trader 时绑定 referral
    bytes calldata data,
    address _trader,
    bytes calldata signature,
    uint256 external_info         // 低 64 位: deadline（非零且超时 → TTSwapError(49)）
) external payable returns (uint256 good1change, uint256 good2change);
```

行为摘要：

- AMM 路径： `g1.buyGoodInput(amount0)` → `g2.buyGoodOutput(ΔV)` ；
- `_goodKey1.toId() == _goodKey2.toId()` 时 revert `TTSwapError(9)` （**无同币快路径**，同币支付请用 `payGood` ）；
- `good1change.amount1() < 1e8` → `TTSwapError(14)` （最小转移价值）；
- `good2change.amount1() < _swapQuantity.amount1()` 且 `amount1 > 0` → `TTSwapError(15)` （滑点）；
- Relayer： `msg.sender != _trader` 时验证 EIP-712；从**输出代币**扣 `executeFee` 记入 `g2.commission[msg.sender]` ；
- 结束后 **仅 g2** 调用 `updateRunBlockConfig()` 。

事件： `e_buyGood(sellgood, forgood, swapvalue, good1change, good2change, trader, external_info)` 。

6.2 `payGood` （Exact-Out 支付 / 同币快路径）

```
function payGood(
    T_GoodKey memory _goodKey1,
    T_GoodKey memory _goodKey2,
    uint256 _swapQuantity,   // 跨币: amount0=max输入, amount1=目标毛输出
                             // 同币: amount1=转账量
    address _recipient,      // 必填 (address(0) → TTSwapError(32))
    bytes calldata data,
    address _trader,
    bytes calldata signature,
    uint256 external_info    // 低 64 位: deadline (超时 → TTSwapError(53))
) external payable returns (uint256 good1change, uint256 good2change);
```

跨币路径 （ `_goodKey1 != _goodKey2` ）：

1. **先**对 good2 执行 `updateRunBlockConfig()` ；
2. `good2change = g2.payGoodOutput(amount1)` → `good1change = g1.payGoodInput(ΔV)` ；
3. 实际输入 `good1change.amount1() + good1change.amount0()` 不得超过 `_swapQuantity.amount0()` （max in）；
4. Relayer 从输出代币扣 `executeFee` （与 `buyGood` 相同计价方式）。

同币快路径 （ `_goodKey1 == _goodKey2` ）：

- 不经 AMM，直接 `transferFrom` → `safeTransfer` 将 `_swapQuantity.amount1` 转给 `_recipient` ；
- Relayer 模式下从输入代币扣 `executeFee` 记入 `commission[msg.sender]` ；
- 适合 X402 商户收款、工资发放等同币场景。

事件： `e_payGood(...)` （跨币与同币字段编码略有差异，见 `I_TTSwap_Market.sol` ）。

6.3 `multicall` （批量调用）

```
function multicall(bytes[] calldata data)
    external payable returns (bytes[] memory results);
```

- 实现 `IMulticall_v4` ，通过 `delegatecall` 批量子调用；
- 修饰器链： `msgValue` + `multicallEntry` （锁级别 1）+ 子函数 `guardedEntry` （1→2）；
- 外层 `msg.value` 在整批调用中只初始化一次 ETH 预算（ `L_Transient` ），适合「授权 + swap + 领佣金」一类编排；
- 注意 run-block 约束：同一 good 在同一 block 槽内仍不可重复写入，批量路径需规划 good 顺序或跨块执行。

6.4 X402 交易协议（签名与执行分离）

EIP-712 类型哈希（Market `Version = "2.0.0"` ）：

buyGood：

```
buyGood(address _trader,address referral,uint256 _goodid1,uint256 _goodid2,
        uint256 _swapQuantity,bytes data,uint256 external_info,uint256 nonce)
```

payGood：

```
payGood(address _trader,address recipient,uint256 _goodid1,uint256 _goodid2,
        uint256 _swapQuantity,uint256 external_info,bytes data,uint256 nonce)
```

- `nonces[_trader]` 在签名验证路径中自增；用户可调用 `cancelNonce()` 作废未用签名；
- Executor 佣金通过 `collectCommission` 批量领取；计价见 `executeFee = 5×10^10` 常量 + 输出 good 现价换算。

6.5 元交易覆盖范围

函数	Relayer / EIP-712	说明
`buyGood`	✅	完整签名验证
`payGood`	✅	完整签名验证
`investGood` / `disinvestProof` / `initGood` 等	❌	`signature` 参数保留 ABI 兼容，**不验证**；`_trader` 必须等于 `msg.sender`
`refreshPromise`	❌	仅 proof owner 可调用，无 meta-tx

7 手续费与分佣

7.1 手续费来源

每笔 swap / invest / disinvest / welfare 都会在池内累积手续费：

- **交易**： `buyFee / sellFee` 从 `swap_fee` 路径累积到 `currentState.amount0` 与 `currentState.amount1`；
- **投资**： `investFee` 按虚拟数量扣除，扣后的实际数量加入 `currentState`；
- **撤资**： `disinvestFee` 从 LP profit 中按比例回拨到 `currentState.amount0`（池内复利）。

LP 净值随池内手续费累积单调递增； `safeLine` 约束保证净值不会被套利者瞬时抽干。

7.2 `allocateFee` 六方分润（协议原生）

撤资时 `L_Good.allocateFee` 按以下顺序分流 profit：

1. **Platform**： `platformFee = getPlatformFee128(profit)`，分到 `commission[address(0)]`（协议金库）；
2. 剩余 `profit -= marketfee` 作为基数，继续分：
   - **LP**： `liquidFee = getLiquidFee(profit)` → `commission[sender]`；
   - **Operator**： `sellerFee = getOperatorFee(profit)` → `commission[owner]`；
   - **Gate**： `gaterFee = getGateFee(profit)` → `commission[gate]`；
   - **Referral**： `referFee = getReferFee(profit)` → `commission[referral]`；
   - **Customer**： `customerFee = getCustomerFee(profit)` → sender 或 gate（取决于 referral 路径）；

分流逻辑存在两条分支：

场景	LP 收到	Operator	Gate	Referral	Customer	Platform
无 referral	`liquidFee + 本金`	—	`operatorFee + customerFee`	—	—	剩余 `profit + marke`
有 referral	`liquidFee + customerFee + 本金`	owner 存在 → `operatorFee`	gate 存在 → `gaterFee`	`referFee`	—	剩余

> **配置约束**： `checkGoodConfig` 强制六项占比（liquid / operator / gate / referral / customer / platform）之和 = 100%。

7.3 佣金领取（Commission）

- 佣金累积在 `S_GoodState.commission` mapping 中；
- 代码保留 **1-unit sentinel**（`commission[addr] = 1`）避免冷 SSTORE，实际可提取金额 = `commission - 1`；
- `collectCommission(address[] goodids, ...)` 单次最多 100 个 good；

- Market Admin 调用 `collectCommission` 时，会批量清空 `commission[address(0)]`（即协议金库）——这是**协议金库回流的唯一链上出口**。

7.4 `goodWelfare` 福利注入

项目方可向池子直接注入代币作为福利：

```
goods[goodid].currentState = add(
    goods[goodid].currentState,
    toTTSwapUINT256(welfare, welfare)
);
```

同时增加 `currentState.amount0` 与 `amount1`，**不发新份额**，直接抬高 LP 净值。受 `currentState.amount0() + welfare > 2**109` 的溢出保护约束。

7.5 首发运营权与运营权拍卖（Operator Lifecycle）

`operatorFee` 虽然只是六方分佣表里的一项（1%–3% 占比），但在生态叙事层面它对应的是一个独立的商业角色：**货架的"品牌方 / 运营者"**。协议围绕这个角色设计了一整条生命周期：

7.5.1 首发者即首任运营者

- `initGood` 的调用者**自动成为该 good 的 `owner`**，全程一次写入、无需额外授权；
- 该 `owner` 地址即 `operatorFee` 的默认收款人（在 `L_Good.allocateFee` 的 `commission[owner]` 分支中自动生效）；
- 同时享有 `modifyGoodByGoodOwner`（调整用户区费率）与 `lockGood`（紧急冻结）权限。

7.5.2 默认 2 年运营期（生态层约定）

协议合约**本身不硬编码"运营期到期自动切换"**——`owner` 字段是静态存储，只要不主动调用 `changeGoodOwner` 就保持现状。但生态层面建议把**2 年**作为首任运营者的默认运营期，理由：

- 与一级市场"IPO 后锁定期"类似，给首发者足够时间变现早期推广投入；
- 2 年也是多数代币从"上线启动"到"达到稳态深度"的典型周期；
- 到期后建议经由"运营权拍卖"市场化转让，让最愿意为货架后续阶段投入的主体接手（见 §7.5.3）。

治理层若希望把"到期自动切换"上链，可在后续版本的合约升级中增加 `ownerExpiresAt` 字段 + 到期后 `changeGoodOwner` 自动走拍卖流程，目前版本以生态约定 + 治理治理建议方式运作。

7.5.3 运营权拍卖：70 / 30 分配

首任运营期到期或原 owner 自愿退出后，货架的运营权通过**拍卖**过渡给下一位运营者。拍卖动作分两段：

1. **链下 / 专用前端拍卖**：标的为"`owner` 身份"——不包括任何供货凭证（`proofs`）或 TTS 算力，后两者与 `owner` 字段解耦；可选英式 / 荷式 / 密封报价，由治理层选择；
2. **链上转交**：拍卖款按协议建议的 **70 / 30** 分配：
   - 70% 归原 `owner`（奖励前期运营投入）；
   - 30% 归社区金库（汇入 `commission[address(0)]`，后续由 Market Admin 通过 `collectCommission` 批量提走，用途由 III 类预算提案决定）；
   - 分账确认后，由 **Market Manager** 调用 `changeGoodOwner(goodid, 新 owner)` 完成身份迁移。

边界约束：

- `changeGoodOwner` 是 Market Manager 权限（§10.3），这意味着拍卖结果的最终落地**受治理约束**，拍卖款分账 / 异议期 / 买家资格等细节需要链下流程 + 链上多签审核；
- 拍卖**只移交 `owner` 字段**，任何 LP 持有的 `S_ProofState` 不受影响，相关 LP 可以继续撤资、领取历史手续费；
- 70 / 30 是**协议层建议比例**，具体实现（托管合约、保证金、定价锚等）随社区治理迭代。首个治理版本可能以一份独立的 `OperatorAuction.sol` 合约承载，或直接由运营团队 + 多签代行。

7.5.4 合约层当前能做什么

当前 v2.0.0 合约直接支持 **`changeGoodOwner` 的权限化转让**：

函数	谁能调用	作用
`changeGoodOwner(goodid, newOwner)`	Market Manager	立即迁移货架的 `owner` 字段

函数	谁能调用	作用
`modifyGoodByGoodOwner`	当下 `owner`	调整用户可调区（power / 费率 / chips）
`lockGood`	`owner` 或 Market Manager	紧急冻结

运营期定时、拍卖 70/30 分账、保证金托管等"运营权市场化"的高阶功能**属于生态层 / 下一代治理合约**，不阻塞当前合约的部署与使用。

8 流动性挖矿（LP → TTS）

每次投资在 `isPromised()` 为真时触发 `L_Proof.stake(TTS_CONTRACT, msg.sender, investvalue)`，相当于把 LP 份额折算成 TTS 算力。

8.1 日增发公式（实测对齐 `_stakeFee`）

```
function _stakeFee() internal {
    if (stakestate.amount0() + 86400 < block.timestamp) {
        stakestate = toTTSwapUINT256(uint128(block.timestamp), stakestate.amount1());
        uint128 leftamount = 2e20 > totalSupply ? 2e20 - totalSupply : 0;
        uint128 mintamount = leftamount < 1e12 ? 1e12 : leftamount / 18250;
        poolstate = add(poolstate, toTTSwapUINT256(ttstokenconfig.getratio(mintamount), 0));
    }
}
```

链上精确语义（相对早期文档表述，已在当前版本中明确）：

- 每 86,400 秒（1 天）触发一次；
- 总可释放量 `leftamount = 2e20 - totalSupply`（TTS 总量上限 = 2e20 wei，按精度 12 即 2 亿 TTS）；
- 未达上限时增发 `leftamount / 18250`（约 50 年线性）；
- **最小保护**：若计算结果 < `1e12`（即 1 TTS），强制取 `1e12`；
- 实际注入量再按 `chainRatio / 10000` 分给当前链的质押池。

8.2 `stake` / `unstake` 核心语义

- 只有 `isCallMintTTS()` 权限的合约可调用（正常部署中 = `TTSwap_Market`）；
- `stake(user, value)`：把用户的 proof value 按当前 `poolstate/stakestate` 比例折算 `netconstruct`，累加到 `stakeproof[keccak256(user, market)]`；
- `unstake(user, value)`：把用户份额兑换为 TTS 收益 `_mint(user, profit)`，profit = 用户份额占当前 poolstate 的比例 - 已记账 construct。

8.3 LP 的三份收益叠加

`disinvestProof` 触发时，LP 单次撤资**同时结算三条现金流**，对应代码里三段独立逻辑：

收益来源	代码路径	计价货币	与价格的关系
① 本金	`L_Good.disinvestGood` 按 `proofInvest1 × shares / proofShares0` 返还	主 Token（`currentgood`）	**与当下池价无关**，严格按 proof 历史数量
② 交易手续费分润	`L_Good.allocateFee` 分 `liquidFee + customerFee` 到 `commission[sender]`	主 Token	与池内累积 `fee` 正相关
③ TTS 挖矿收益	`L_Proof.unstake`（当 `disinvestTTSValue > 0`）	**TTS**	与 `poolstate` 累积增发 × 用户算力占比正相关

> 三者由同一笔 `disinvestProof` 调用合并转账，LP 无需分别签名 / 分别领取。部分撤资时三项均按撤出份额比例兑现，剩余部分继续挖矿。

对照关系：

- 本金保障对应 §5.4 的"三重防御 + 零无常损失"；
- 手续费分润对应 §7.2 的六方分账（LP 在绝大多数配置下占 `liquidFee` 50%–80% 的最大份额）；
- TTS 收益对应 §8.1–8.2 的 stake/unstake 会计学。

相较 Uniswap 系列仅有"本金+IL 修正"、Curve 仅有"本金+CRV 外部排放"，TTSWAP 的 LP 在一次撤资调用里同时完成**本位货币复利 + 六方分佣 + 股权型代币**三档结算，是协议原生支持的三重收益模型。

9 代币经济（TTS Tokenomics）

9.1 基本信息（ `TTSwap_Token.sol` ）

项	值	代码位置
名称 / 符号	TTSwap Token / **TTS**	构造函数
精度	**12 位小数**	`ERC20("TTSwap Token", "TTS", 12)`
最大供应量	**2 亿 TTS** = `2e20` wei	`_stakeFee` 中的 `2e20`
`left_share`	初始 **4.5e19**（按 12 位精度即 **45,000,000 TTS**）	`uint128 public override left_share = 45_000_000_000_000_000_000;`
公售硬顶	**250,000 USDT** = `2.5e11` （按 6 位精度）	`publicSell` 中 `publicsell > 2.5e11` 回滚
Executor 补偿	`executeFee = 5e10` = **0.05 USDT**	常量
版本	**1.16.0**	`string constant Version`
跨链	`ttstokenconfig.ismain()` 判定主链	`onlymain` 修饰器

9.2 价格翻倍解锁机制（完全链上化）

下文明确列出链上代码的实现路径：

分配阶段（ `addShare` / `permitShare` ）：

`s_share { leftamount, chips, metric }`：

- `leftamount`： 剩余可解锁量；
- `chips`： 总份数（每次解锁 1/chips）；
- `metric`： 已解锁次数计数器（每次 `shareMint` 后 +1）。

解锁阶段（ `shareMint` ）：

```
function shareMint() external onlymain {
    if (!I_TTSwap_Market(marketcontract).ishigher(
            address(this), usdt,
            2 ** shares[msg.sender].metric * 2 ** 128 + 20_000_000
    )) revert TTSwapError(68);
    if (shares[msg.sender].leftamount == 0) revert TTSwapError(69);
    uint128 mintamount = shares[msg.sender].leftamount / shares[msg.sender].chips;
    shares[msg.sender].leftamount -= mintamount;
    shares[msg.sender].metric += 1;
    _mint(msg.sender, mintamount);
}
```

核心链上语义：

- 每次 `shareMint` 前调用 `TTSwap_Market.ishigher(TTS, USDT, priceThreshold)` 做**价格触发检查**；
- 价格阈值 = $2^{\text{metric}} \times 2^{128} + 20{,}000{,}000$，即**每解锁一次阈值翻倍**；
- 单次解锁量 = `leftamount / chips` （ `chips` 越大，单次解锁越少）；
- **币价没涨到阈值 → `ishigher` 返回 false → `shareMint` 回滚**，解锁永远不会发生。

4C 角色占比通过在 `addShare` 时给不同接收人发不同 `leftamount / chips` 配比实现：

4C 角色	分配总额占比	`chips` 参考值（单次上限）	触发示例
Founder	20%	`chips = 20` → 单次 1/20 (5%)	涨 1,000× 累计解锁约 37%
Partner	12%	`chips = 14` → 单次 1/14 (7.1%)	涨 400× 累计约 45%
Value	44%	`chips = 12` → 单次 1/12 (8.3%)	涨 200× 累计约 50%
Capital	24%	`chips = 8` → 单次 1/8 (12.5%)	涨 100× 累计约 60%

9.3 `permitShare` 异步批量分配

Token Admin 可对一份 `s_share` 结构离线 EIP-712 签名，签名字段：

```
permitShare(
    uint128 amount,
    uint120 chips,
    uint8  metric,
    address owner,
    uint128 existamount,
    uint128 deadline,
    uint256 nonce
)
```

接收方调用 `permitShare(share, deadline, signature, signer)` 上链自领份额，适合**大规模空投 / 合作方配额 / 生态激励**：批量、异步、零 Gas 签名。

9.4 公售阶梯定价（链上硬顶）

`publicSell(usdtamount, data)` 按三档比例铸造 TTS：

Tier	已募资 USDT 区间	单价	合约常量
1	≤ 87,500	**1 USDT = 25 TTS**	`usdtamount × 25_000_000`
2	87,500 < x ≤ 162,500	**1 USDT = 20 TTS**	`usdtamount × 20_000_000`
3	162,500 < x ≤ 250,000	**1 USDT = 16 TTS**	`usdtamount × 16_000_000`

> 硬顶 250,000 USDT 超过则回滚 `TTSwapError(70)`；USDT 由 Token Admin 通过 `withdrawPublicSell` 提取到指定地址。

9.5 通胀与通缩

通胀（供给侧）：

$$\text{daily_mint} = \frac{2 \times 10^{20} - \text{totalSupply}}{18{,}250} \times \frac{\text{chainRatio}}{10{,}000}$$

约 50 年线性释放，全部分给 LP 挖矿。

通缩（需求侧）：

- `commission[address(0)]`（协议金库）由 Market Admin 通过 `collectCommission` 批量领取；
- 社区金库通过**二级市场回购 TTS 并销毁**，形成通缩引擎；
- `burn(value)` 用户自主销毁 + DAO Admin `burn(from, amount)` 兜底。

9.6 DAO 紧急兜底

合约保留 DAO Admin 专属：

- `mint(to, amount)`：合约迁移补偿 / 紧急修复（不受 `left_share` 与 `publicSell` 限制）；
- `burn(from, amount)`：强制销毁（合约迁移 / 漏洞兜底）；

所有操作均触发 `Transfer` 事件，社区链上可审计。

10 协议治理与角色系统

10.1 `UserConfig` 位图（256-bit）

Bit	字段	语义
255	DAO Admin	最高权限， `setDAOAdmin / setTokenAdmin / setMarketAdmin / setStakeAdmin / setEnv / disableUpgrade / mint / burn /`
254	Token Admin	`setTokenManager / setCallMintTTS / setRatio / addShare / burnShare / withdrawPublicSell / permitShare (signer)`
253	Token Manager	`setBan`
252	Market Admin	`modifyGoodByAdmin / collectCommission(protocol)`
251	Market Manager	`modifyGoodByManager / changeGoodOwner / lockGood / freezeMarket`
250	CallMintTTS	`stake / unstake` 权限（授予 Market 合约）
249	Stake Admin	`setStakeManager`
248	Stake Manager	流动性挖矿运营
160	Ban 标志	被封禁地址，在撤资时自动丧失 gate/referral 资格
0–159	Referral 地址	用户的推荐人地址

关键权限链： `DAOAdmin → {TokenAdmin, MarketAdmin, StakeAdmin} → 对应 Manager` 。

10.2 `goodConfig` 三层修改隔离（v2.0.0）

函数	调用权限	可改区段	用途
`modifyGoodByGoodOwner`	**Good Owner**	用户区（bits 166–128：power / 费率 / chips）	项目方日常调参
`modifyGoodByManager`	**Market Manager**	管理员区（bits 246–191：分佣 / safeLine / isPromise 等）	管理员介入， `checkGoodConfig` 校验
`modifyGoodByAdmin`	**Market Admin**	超级管理员区（bits 255–247： `isValueGood` / ERC 类型）	最高级别提权

10.3 Token 管理工具

函数	权限	说明
`lockGood`	Token Owner or Market Manager	即时冻结某个 good 的交易（ `isFreeze = true` ）
`changeGoodOwner`	Market Manager	迁移 good 所有权
`cancelNonce`	任何用户	自增 `nonces[_trader]` 作废未使用的签名
`queryCommission`	任何用户（view）	批量查询多个 good 对某地址的累积佣金
`collectCommission`	任何用户 + Market Admin（后者可收协议金库）	批量领取多个 good 的 commission

10.4 生态收益角色

角色	典型分成	机制
LP	50–80%	`liquidFee → commission[sender]`
Token Owner	1–3%	`operatorFee → commission[owner]`
Gate（前端 / 聚合器）	5–25%	`gaterFee → commission[gate]`
Referral	5–10%	`referFee → commission[referral]`
Customer	-10%（折扣回拨）	`customerFee → commission[sender]` 或 gate
Platform	2–8%	`platformFee → commission[address(0)]`（社区金库）
Executor	固定 `executeFee`	X402 代付补偿

10.5 三权分立治理架构（社区 / CEO / 董事会）

`UserConfig` 的位图只定义了合约层面的"钥匙"。在治理架构层面，这些钥匙被映射到三个相互独立的角色，形成"**社区决策 + CEO 执行 + 董事会监督**"的三角制衡结构：

治理角色	对应合约钥匙	产生方式	核心职责
社区（DAO）	无直接钥匙，通过治理合约 executor 调用	任一 TTS 持有人均可参与	选举 CEO、授权董事席位、通过所有 I–IV 类提案
CEO（执行层）	**Market Admin + Token Admin**（`UserConfig[252] / [254]`）	社区 I 类提案选举，可随时罢免	组阁研发 / 市场 / 运营团队、持有业务钥匙、紧急响应（`freezeMarket` / `freezeToken`）
董事会（监督层）	**DAO Admin**（`UserConfig[255]`）	由**合伙人 A 类型 TTS 持有者**组建多签	`disableUpgrade` / 紧急 `mint/burn` / 任命下级 Admin；链下负责监督 + 仲裁 + 价值观复议

制衡要点：

1. **决策权与执行权分离**：CEO 无权单独修改任命、挪用金库或启动 `disableUpgrade`，所有组织人事 / 协议变更 / 预算 / 重大事项必须走社区提案；
2. **董事会独立于 CEO**：董事会持 DAO Admin 钥匙但**不持任何 Market/Token Admin 钥匙**；CEO 无权任命或撤销董事；董事会对违反核心价值观（零无常损失、一币一池、六方分账总和 = 100%、用户资金可取出、`disableUpgrade` 不可逆）的提案可发起**复议动议**，原提案进入 72h 冷静期并需社区重新投票；
3. **紧急按钮与永久决议分离**：`freezeMarket / freezeToken / lockGood` 在 CEO 团队手里（秒级响应、可回滚）；`disableUpgrade` 在董事会手里（永久不可逆、需社区 IV 类提案 + Timelock 授权）；
4. **外部参与者不受治理约束**：供货商、顾客、推荐人、货架 owner 的资金取出能力由合约硬编码保障，任何治理决议都无权阻断 `disinvestProof` / `buyGood` / `collectCommission`。

提案分类（决议层面）：

类别	典型场景	建议通过门槛	Timelock
I 组织人事	提名 / 罢免 CEO、授权 Admin 钥匙	过半赞成	短（24–48h）
II 协议变更	`modifyGoodByAdmin` 提权价值 Token、`upgrade` 到 V2、调整六方分账上限	过半赞成	中（48–72h）
III 预算财务	`collectCommission(protocol)` 使用、`withdrawPublicSell` 提款、`addShare/permitShare` 批量分配	过半赞成	中（48–72h）
IV 重大事项	`disableUpgrade`、紧急 `mint/burn`、治理合约参数变更	超级多数（≥ 2/3）	长（≥ 7 天）

> **实施阶段**：协议上线初期以 "链下多签 + 链上关键点 hash 存证" 的半链上模式运作；中期引入治理合约 + Timelock；远期由独立的董事会合约模块承载"复议锁 + 自动 executor"，最终在 `disableUpgrade` 执行后 DAO Admin 钥匙永久失效。

合约层面的 `setDAOAdmin / setTokenAdmin / setMarketAdmin / setStakeAdmin` 是这套三角架构的唯一上链入口：**每把钥匙都是"工作钥匙"，随社区提案动态增删**，不是任何个人的私产。

11 合约升级与紧急熔断

TTSWAP 采用 **Delegatecall Proxy** 模式。生产部署中用户与集成方应对接 **Proxy 地址**（ `TTSwap_Market_Proxy` / `TTSwap_Token_Proxy` ），逻辑合约（ `TTSwap_Market` / `TTSwap_Token` ）仅作 `implementation` 指针目标。

11.1 Proxy 架构

合约	角色	关键状态
`TTSwap_Market_Proxy`	市场状态容器 + 转发	`implementation` ， `TTS_CONTRACT` （immutable） ， `nonces` ， `upgradeable`
`TTSwap_Market`	逻辑实现	`goods` ， `proofs` ， `executeFee` ； 保留与 Proxy 对齐的 `implementation` / `upgradeable` 占位槽
`TTSwap_Token_Proxy`	代币状态容器 + 转发	`balanceOf` ， `allowance` ， `userConfig` ， `stakestate` ， `left_share` ， `implementation`
`TTSwap_Token`	逻辑实现	`shares` ， `stakeproof` ， `marketcontract` ， `poolstate`

部署顺序（推荐）：

1. 部署 `TTSwap_Token` 逻辑 → `TTSwap_Token_Proxy` （DAO Admin、 `ttstokenconfig` 、元数据）；
2. 部署 `TTSwap_Market` 逻辑 → `TTSwap_Market_Proxy` （绑定 `TTS_CONTRACT` ）；
3. DAO 调用 `setEnv(market_proxy)` ； Token Admin 授予 Market `setCallMintTTS` 与 Market Admin/Manager 角色。

11.2 升级与熔断

函数	所在合约	权限	用途
`upgrade(address newImpl)`	Market/Token Proxy	Market/Token Admin，且 `upgradeable == true`	热升级逻辑合约地址
`disableUpgrade()`	Market Proxy	**DAO Admin**	永久关闭升级能力（ `upgradeable = false` ）
`freezeMarket()`	Market Proxy	Market Manager，且 `upgradeable == true`	紧急熔断： `implementation = address(0)`
`freezeToken()`	Token Proxy	Token Manager，且 `upgradeable == true`	紧急熔断代币逻辑

> **两权分离**：升级权在 Admin（日常运维），永久锁定权在 DAO Admin（治理终局），熔断权在 Manager（快速响应）。Proxy `fallback` 在 `implementation == address(0)` 时 revert `TTSwapError(63)` 。

12 主要函数与链上 Gas

> Gas 数据来自 v2.0.0 快照基准；v2.0.0 swap 引擎改为分片迭代，实际 gas 可能略有变化，部署后应以 `.forge-snapshots/` 复测为准。

12.1 合约版本

合约	版本	部署形态
TTSwap_Market	**2.0.0**	Proxy + Logic
TTSwap_Token	**1.16.0**	Proxy + Logic
TTSwap_Market_Proxy / TTSwap_Token_Proxy	—	用户入口地址

12.2 单次调用 Gas

函数	稳态 gas	说明
`buyGood` ERC20 → ERC20	~65k	含 `buyGoodInput/Output` 分片（v2 快照基准）

函数	稳态 gas	说明
`payGood` ERC20 跨币	~70–90k	exact-out 逆路径
`buyGood` NativeETH → ERC20	~72k	无 ERC20 入转账
`investGood` 单边追加	99–120k	按池内价自动计价
`disinvestProof`	128–184k	含六方分润 + 条件 unstake
`initGood`	~400k	首次开池 + proof 初始化（无 stake）
`multicall` (2× buyGood)	~130k+	取决于子调用数量
`goodWelfare`	~61k	
`collectCommission`（单 good）	~25k	

12.3 与主流协议对比（完整 swap tx）

协议	完整 swap tx gas	结构特征
TTSwap	**~93k**	Singleton + 单次 call + 无 callback
Uniswap V4	130–180k	`unlock → unlockCallback → sync/settle/take` 多段
Uniswap V3	~127k	SwapRouter + NFPM
Uniswap V2	~102k	Router + Pair
Curve	115–135k	Vyper + 迭代 D
1inch Fusion+	200–400k	聚合路由

结构性节省来源：无 `unlock/lock` 外壳（-5k–10k）、无 Router 层、无 Permit2 双签、无 `sync/settle/take` 三段式、proof 为 mapping 而非 ERC721（省 ~50–80k NFT mint）。

13 典型场景

场景	核心函数	推荐度
长尾代币启动（Launchpad / IDO）	`initGood + investGood + isPromised + refreshPromise`	★★★★★
单边做市（零 IL）	`investGood + disinvestProof`	★★★★★
跨币 swap（指定输入）	`buyGood`	★★★★★
跨币支付（指定输出）	`payGood`	★★★★★
同币 X402 收款	`payGood`（同 good 快路径）	★★★★★
Relayer 代付 gas	`buyGood / payGood` + EIP-712	★★★★
批量编排（swap + 领佣）	`multicall`	★★★★
价值 Token 部署	`initGood + modifyGoodByAdmin(isValueGood)`	★★★★★
B2B2C 聚合分销	六方分佣 + `setReferral`	★★★★★
股票化代币 / RWA 上架（路线图）	`T_GoodKey(ercType=2) + initGood + buyGood`	★★★★（待开启 ERC-1155 分支）
预测市场结果份额交易（路线图）	`T_GoodKey(ercType=3) + initGood + buyGood`	★★★★（待开启 ERC-6909 分支）
稳定币低滑点互换	`buyGood` + 调大 M（`power`）	★★★
高频套利（聚合器接入）	`multicall` 批量 `buyGood / payGood`（注意 run-block 槽）	★★★

14 联系方式

- **Twitter**: @ttswapfinance
- **Telegram**: @ttswapfinance
- **Email**: bussiness@ttswap.io
- **Discord**: ttswap
- **Website**: ttswap.io
- **GitHub**: github.com/ttswap

15 法律许可

文件类别	License
`src/TTSwap_Market.sol` / `TTSwap_Market_Proxy.sol` / `TTSwap_Token.sol` / `TTSwap_Token_Proxy.sol`	BUSL-1.1 （商业源使用许可）
`src/libraries/L_Good.sol` / `L_Proof.sol`	BUSL-1.1
`src/base/ERC20.sol`	AGPL-3.0-only
`src/interfaces/*.sol`	MIT
`src/libraries/L_Currency.sol` / `L_Error.sol` / `L_GoodConfig.sol` / `L_UserConfig.sol` / `L_TTSTokenConfig.sol` / `L_TTSwapUINT256.sol` / `L_SignatureVerification.sol`	MIT
`src/libraries/L_Transient.sol`	GPL-3.0-or-later
`src/interfaces/IWETH9.sol`	GPL-2.0-or-later

详细版权信息以 GitHub 仓库中的 `LICENSE` 文件为准。核心业务逻辑采用 BUSL-1.1 保护，接口与工具库采用 MIT 许可便于社区集成。

附录 A：价值守恒交易模型核心公式详细推导

TTSWAP 的交易过程分两步走。兑换内核把对偶系数**硬编码为常量 2**.

A.1 第一步：输入侧 → 转移价值 ΔV

瞬时价格：

- 交易前：$P_{\text{before}} = V_A / Q_A$；
- 交易后（假设）：$P_{\text{after}} = V_A / (Q_A + \Delta a)$。

协议以调和平均的"执行密度"计算平均单价（Δa 为扣除 `sellFee` 后的净输入，代码按 `current_quantity / 100` 分片迭代累加，见 §3.1）：

$$\Delta V = \frac{2 \cdot V_A \cdot \Delta a}{2 \cdot Q_A + \Delta a}$$

A.2 第二步：输出侧 → 应释放的代币数量 Δb

输出侧用**同一个常量 2**把转移价值映射回数量，无对偶变换、无 `getK2`：

$$\Delta b = \frac{2 \cdot Q_B \cdot \Delta V}{2 \cdot V_B + \Delta V}$$

代码按 `current_value / 100` 分片迭代累加 Δq_i，循环结束后再用 `getBuyFee` 扣输出侧手续费（`L_Good.buyGoodOutput`）。

A.3 TTSwap的可调维度

- **M（流动性放大，`getPower / getLimitPower`）**：通过虚拟量放大池深度，稳定币 / 高相关资产配置较大 M 即可把平衡区曲线压平，逼近恒定和。

- **`safeLine`（`getSafeLineUpper/Lower`）**：约束单池 Q 的可交易区间，承担"偏离即提摩擦 / 阻断"的角色，越界 revert `TTSwapError(55/56)`。
- **费率（buy / sell / invest / disinvest）**：逐币万分之一精度独立配置。
- **O(1) 代数**：全程加减乘除，无指数、无迭代、无外部库调用。

A.4 与 CPMM 的衔接

公式即标准调和平均：

$$\Delta V = \frac{2V_A\Delta a}{2Q_A + \Delta a}, \quad \Delta b = \frac{2Q_B\Delta V}{2V_B + \Delta V}$$

当 $V_A = V_B$ 时退化为 $x \cdot y = k$（见附录 B 的 CPMM 等价性）。

附录 B：CPMM 等价性证明（$V_A = V_B$）

设 $V_A = V_B = V$，$2 = K_B = 2$。代入退化公式并联立：

$$\Delta V = \frac{2V\Delta a}{2Q_A + \Delta a}, \quad \Delta b = \frac{2Q_B\Delta V}{2V + \Delta V}$$

代入化简（分子分母除以 $2V$，整理分母为 $\frac{2(Q_A+\Delta a)}{2Q_A+\Delta a}$）：

$$\Delta b = \frac{Q_B \cdot \Delta a}{Q_A + \Delta a}$$

即 Uniswap V2 的标准输出公式 $\Delta y = \frac{y\Delta x}{x+\Delta x}$。**证毕**。

这说明 TTSWAP 在 $V_A = V_B$ 时拥有与 Uniswap 相同的市场深度与定价特性；而 $V_A \neq V_B$ 时自然扩展为加权池（附录 C）。

附录 C：Balancer 等价性证明（$V_A \neq V_B$）

C.1 Balancer 回顾

加权恒定乘积：$(Q_A)^{W_A} \cdot (Q_B)^{W_B} = k$，spot price：

$$SP_{\text{Balancer}} = \frac{Q_A \cdot W_B}{Q_B \cdot W_A}$$

C.2 TTSWAP 边际价格

对输入侧公式求 $\Delta a \to 0$ 的极限（系数恒为 2）：

$$dV = \frac{V_A}{Q_A}da, \quad db = \frac{Q_B}{V_B}dV$$

联立得：

$$SP_{\text{TTSWAP}} = \frac{da}{db} = \frac{Q_A \cdot V_B}{Q_B \cdot V_A}$$

C.3 对照结论

映射 $V \leftrightarrow W$，两者的 spot price **完全一致**。大额交易下 TTSWAP 的调和平均曲线与 Balancer 的指数曲线在凸性、渐近线、滑点方向上高度拟合，但 TTSWAP **无需任何指数运算**，Gas 消耗是 $O(1)$ 常数。

附录 D：Curve StableSwap 等价性证明

D.1 Curve 回顾

恒定和 + 恒定乘积组合：

$$A \cdot n^n \cdot \sum x_i + D = ADn^n + \frac{D^{n+1}}{n^n \prod x_i}$$

通过放大系数 A 让稳定币区滑点 < 1 bp。但求解 D 需要 Newton-Raphson 迭代。

D.2 TTSWAP 的双参数平衡态近似

StableSwap 式低滑点主要由 **M（流动性放大）** 实现，偏离回归由 **`safeLine` 边界** 承担：

- **M（流动性放大， `getPower` ）**：对稳定币调大 M，使平衡区的曲线接近直线（恒定和）；
- **`safeLine` （ `getSafeLineUpper/Lower` ）**：监控 Q 偏离基准库存 `I + virtual` 的程度，越界即 revert（ `TTSwapError(55/56)` ），强制后续交易反向把 Q 拉回。

稳定币平衡态数学：当 $V_A = V_B = V$、$Q_A = Q_B = Q$、M 较大（池内 virtual quantity 主导）时：

$$\Delta V \approx \frac{V_A \Delta a}{Q_A}, \quad \Delta b \approx \frac{Q_B \Delta V}{V_B} = \Delta a$$

即 $\Delta a \approx \Delta b$，1:1 兑换，滑点极小。

偏离态回归：当交易把 Q 推偏（$Q \ll I$ 或 $Q \gg I$），调和均值曲线自身的滑点随偏离非线性放大，叠加 `safeLine` 上下界在触及边界时直接 revert（ `TTSwapError(55/56)` ），共同推动后续交易反向把 Q 拉回 `I + virtual` 附近。这种"**偏离即自动提摩擦 / 到界即阻断**"的机制等价于 Curve 在 $x_i \neq x_j$ 时从"恒定和主导"过渡到"恒定乘积主导"的软切换。

D.3 工程优势

维度	Curve	TTSWAP
计算复杂度	迭代求 D + 高次幂	加减乘除 + O(1)
Gas 成本	较高	极低（92,747 稳态）
适用范围	稳定币对	通用代币，M 与费率可调（K 固定为 2）
参数灵活度	治理缓慢调整 A	每代币独立配置 M、 `safeLine` 、费率
偏离态响应	A 固定，曲线自动向乘积过渡	曲线自然滑点 + `safeLine` 边界阻断驱动 Q 回归
平衡态低滑点	A 调大	M 调大（正交维度）

附录 E： `goodConfig` 256-bit 布局（完整）

```
Bit 255           isvaluegood         1 bit
Bits 254–247      erctype             8 bits
Bit 246           isFreeze            1 bit
Bit 245           isVerified          1 bit
Bit 244           isPromise           1 bit
Bits 243–241      liquidFee           3 bits   × 0.1   default:6
Bits 240–237      operatorFee         4 bits   × 0.02  default:1
Bits 236–234      gateFee             3 bits   × 0.04  default:5
Bits 233–229      referFee            5 bits   × 0.01  default:8
Bits 228–224      customerFee         5 bits   × 0.01  defualt:8
Bits 223–219      platformFee         5 bits   × 0.01  default:2
Bits 218–214      limitPower          5 bits   × 1
Bits 213–204      safeLine            10 bits
Bits 203–192      contractType        12 bits
Bit 191           reversed            1 bit
Bits 190–179      getLastRun          12 bits
Bits 178–167      reversed           12 bits  × 10
Bits 166–162      power               5 bits   × 1
Bits 161–154      disinvestChips      8 bits   x 4
Bits 153–148      investFee           6 bits   × 0.0001  defualt:0.08%
Bits 147–142      disinvestFee        6 bits   × 0.0001  defualt:0.08%
Bits 141–135      buyFee              7 bits   × 0.0001  defualt:0.08%
Bits 134–128      sellFee             7 bits   × 0.0001  defualt:0.08%
Bits 127–0        marketValue field   128 bits
```

六方分佣和约束： `checkGoodConfig` 强制 `liquidFee + operatorFee + gateFee + referFee + customerFee + platformFee = 100%` （以 10% / 2% / 4% / 1% / 1% / 1% 为单位）。

精度校验函数：

- `getPower(config)`： `config[166..162] × 100`，默认 100（即 1×）；
- `getLimitPower(config)`： `config[218..214] × 100`，投资杠杆上限；
- `getSafeLineUpper/Lower(config, amount)`： bits 213–204，默认 80；
- `getInvestFee / getDisinvestFee / getBuyFee / getSellFee`： 万分之一精度，默认均为 8（0.08%）。

附录 F：投资与撤资完整计算流程（代码对齐）

F.1 初始状态

项	值
$V =$ investState.amount1	40,000
$Q =$ currentState.amount1	40,000
$I =$ currentState.amount0	40,000
$S =$ investState.amount0	40,000
$M =$ getPower	5

F.2 投资 200 单位（ `investGood` 核心流程）

1. `enpower = getInvestPower(goodid)` （按 Q/I 动态缩放，上限 `getPower()` ）；
2. `investFeeQuantity = getInvestFee(200)` （本例假设 = 0）；
3. `investQuantity = (200 − 0) × enpower / 100` （虚拟数量）；
4. `investValue = poolPrice × investQuantity`，经 `getInvestThreshold` 截断；
5. `investShare = goodShares / goodInvestQuantity × 200` ；

6. 更新池： `currentState` 、 `investState` 、 `goodConfig.amount1` （虚拟量）；
7. 更新 proof：shares / state / invest；
8. `L_Proof.stake(TTS, msg.sender, investvalue_actual)` 发放 TTS 算力。

F.3 市场变动后（假设池子出现收益）

- V = 41,000（不变）
- Q = 41,000 → **44,000**
- I = 40,200 → **43,000**
- S = 40,200

F.4 撤资 100 份（ `disinvestProof` 核心流程）

1. 身份校验： `S_ProofKey(_trader, currentgood).toId() == _proofid` ；
2. 按 proof 历史比例算返还：
   - `actualDisinvestQuantity = proofInvest1 × goodshares / proofShares0 = 200 × 100 / 200 = 100` ；
   - `virtualDisinvestQuantity = proofInvest0 × 100 / 200 = 500` ；
3. 按当前池计算当前价值：
   - `profit = currentState.amount0 / investState.amount0 × shares = 43000/41000 × 100 ≈ 104.88` ；
4. 撤资费： `actual_fee = getDisinvestFee(virtualDisinvestQuantity)` （假设 0.5%，= 2.5）；
5. 池子更新：
   - `currentState -= (profit, virtualDisinvestQuantity + profit - actualDisinvestQuantity)` ；
   - `investState -= (shares, disinvestvalue.amount0)` ；
   - `currentState += (actual_fee, actual_fee)` （撤资费回池）；
6. 净 profit = `profit - actualDisinvestQuantity = 104.88 - 100 = 4.88` ；
7. `allocateFee(profit=4.88, gate, referral, quantity=100, sender)` 六路分佣；
8. `L_Proof.unstake(TTS, sender, divestvalue.amount1())` 。

F.5 Disinvest Chips 限流

若 `virtualDisinvestQuantity > getDisinvestChips(currentState.amount1)` → revert `TTSwapError(27)` ；若 `disinvestvalue.amount0 > getDisinvestChips(investState.amount1) || disinvestvalue.amount0 < 10000` → revert `TTSwapError(26)` 。

附录 G：代币兑换完整计算流程

G.1 初始状态

- Token A：V=40000, Q=20000, price=2.0, fee=0.01%;
- Token B：V=20000, Q=40000, price=0.5, fee=0.01%;

G.2 用户输入 2500 Token A（ `buyGood` ERC20→ERC20）

1. `sellFee = getSellFee(2500)` ；
2. `Δa (net) = 2500 - sellFee` ；
3. Step 1（ `buyGoodInput` ，分片迭代）：
   每片 $\Delta V_i = \frac{2V\Delta q_i}{2Q + \Delta q_i}$，累加得总 ΔV；
4. Step 2（ `buyGoodOutput` ，分片迭代）：
   每片 $\Delta q_i = \frac{2Q\Delta V_i}{2V + \Delta V_i}$，累加得总输出；
5. 输出费 `buyFee` ，用户实收 = 毛输出 − `buyFee` 。

G.3 状态更新

- Token A: `currentState` amount0 += 0.25, amount1 += 2500
- Token B: `currentState` amount0 += 0.84, amount1 -= (8420.3 - 0.84)
- `investState` 均不变

G.4 价格变化

- Token A 新价 ≈ 40000 / 22500 ≈ 1.78（下跌）
- Token B 新价 ≈ 20000 / 31580 ≈ 0.63（上涨）

附录 H：手续费计算与分配

H.1 累积公式

每笔 swap：

$$I_{\text{new}} = I_{\text{old}} + \text{fee}$$

净值：

$$\text{NetValue} = \frac{I + \text{AccumulatedFees}}{S}$$

随交易量增长，净值单调递增。

H.2 `allocateFee` 分流伪代码

```
platformFee = getPlatformFee128(profit);
profit -= platformFee;

liquidFee   = getLiquidFee(profit);
operatorFee = getOperatorFee(profit);
gateFee     = getGateFee(profit);
referFee    = getReferFee(profit);
customerFee = getCustomerFee(profit);

if (referral == 0) {
    if (gate == 0) {
        commission[0]       += (profit - liquidFee + platformFee);
        commission[sender]  += (liquidFee + actualDivestQuantity);
    } else {
        commission[sender]  += (liquidFee + actualDivestQuantity);
        commission[gate]    += (operatorFee + customerFee);
        commission[0]       += (profit + platformFee - liquidFee - operatorFee - customerFee);
    }
} else {
    if (owner != 0) commission[owner]  += operatorFee;
    else            platformFee         += operatorFee;
    if (gate  != 0) commission[gate]    += gateFee;
    else            platformFee         += gateFee;
    commission[referral] += referFee;
    commission[0]        += platformFee;
    commission[sender]   += (liquidFee + customerFee + actualDivestQuantity);
}
```

H.3 佣金领取 1-unit sentinel

合约保留 1 最低位规避冷 SSTORE：

```
if (tranferamount > 1) {
    goods[normalgood].commission[msg.sender] = 1;
    normalgood.safeTransfer(msg.sender, tranferamount - 1);
}
```

附录 I：反向兑换闭环性

设 V_A, Q_A, V_B, Q_B，用户输入 Δa 换出 Δb：

1. 正向：$\Delta V = \frac{2V_A\Delta a}{2Q_A+\Delta a}$，$\Delta b = \frac{2Q_B\Delta V}{2V_B+\Delta V}$；
2. 反向输入 Δb：

$$\Delta V' = \frac{2V_B\Delta b}{2(Q_B-\Delta b)+\Delta b} = \frac{2V_B\Delta b}{2Q_B-\Delta b}$$

3. 由正向 Δb 公式变形：$\Delta V = \frac{2V_B\Delta b}{2Q_B-\Delta b}$，可得 $\Delta V' = \Delta V$（**价值守恒**）；
4. 反向输出 $\Delta a_{\text{out}} = \frac{2(Q_A+\Delta a)\Delta V}{2V_A+\Delta V}$；
5. 代入 $\Delta V(2Q_A+\Delta a) = 2V_A\Delta a$ 化简得：$\Delta a_{\text{out}} = \Delta a$。

证毕。路径无关，不存在无风险套利。

附录 J：拆分交易误差

定义：$\Delta a = x_1 + x_2$，比较一次 vs 两次。

误差公式（忽略手续费）：

$$\text{Diff} = \frac{C_1(x_1+x_2)(V_A^2-V_B^2)x_1x_2}{(C_2+N(x_1+x_2))(C_2+Nx_1)(C_2+2V_Bx_1+Nx_2)}$$

其中 $C_1 = 2Q_BV_A$，$C_2 = 2V_BQ_A$，$N = V_A+V_B$。

J.1 路径无关条件

- $V_A = V_B$：Diff = 0（严格路径无关）；
- $V_A > V_B$：Diff > 0（一次性交易更优）；
- $V_A < V_B$：Diff < 0（拆分交易更优）。

J.2 误差量级

$$\frac{\text{Diff}}{\Delta b} \approx \left(\frac{\Delta a}{Q_A}\right)^2 \cdot \frac{V_A^2-V_B^2}{V_B^2}$$

单笔 $\Delta a/Q_A \approx 10^{-3}$ 时，相对误差 $\sim 10^{-6}$，工程可忽略。

附录 K：三角闭环证明

$A \to B \to C \to A$ 的三跳循环：

K.1 价值引理

每跳产生的 ΔV 与前一跳的 ΔV 严格相等（由 Δb 公式的可逆变形得到）：

$$\Delta V_{AB} = \Delta V_{BC} = \Delta V_{CA} = \Delta V$$

K.2 状态还原

- Token B 池：$Q_B + \Delta b$ 后出 Δb，回到 Q_B；
- Token C 池：$Q_C - \Delta c$ 后入 Δc，回到 Q_C；
- Token A 池：$Q_A + \Delta a$ 后出 Δa_{final}。

K.3 证明 $\Delta a_{\text{final}} = \Delta a$

$$\Delta a_{\text{final}} = \frac{2(Q_A + \Delta a)\Delta V}{2V_A + \Delta V}$$

代入 $\Delta V(2Q_A + \Delta a) = 2V_A \Delta a$ 展开化简得等式成立。**证毕**。

Token A 池状态也随之还原，系统回滚，无无风险套利空间。

白皮书结束 · TTSWAP · 2026-05-09